

1. Det store bildet: hvordan hele pensum henger sammen

Hele faget kan forstås som spørsmålet: **Hvordan utvikler vi programvare som er nyttig, korrekt, brukbar, trygg, testbar, vedlikeholdbar og forsvarlig?**

De forskjellige delene av pensum svarer på ulike deler av dette:

- **MMI / interaksjonsdesign / prototyping / evaluering / brukertesting / universell utforming / WCAG** handler om: Lager vi riktig løsning for mennesker?
- **UML / brukerhistorier / domenemodell / use case / sekvens / aktivitet / klasse / tilstand** handler om: Hvordan beskriver og kommuniserer vi løsningen før og under utvikling?
- **Programvarearkitektur / arkitekturmønstre / REST / deployment** handler om: Hvordan bygger vi løsningen teknisk slik at den fungerer og kan utvikles videre?
- **Systemutviklingsmetodikk / smidig / Scrum / Kanban / XP / TDD / BDD / DDD / DevOps** handler om: Hvordan organiserer vi arbeidet for å få framgang og kvalitet?
- **Testing** handler om: Hvordan verifiserer og validerer vi at systemet virker og oppfyller behovene?
- **GDPR / profesjonsetikk / lovkrav for universell utforming** handler om: Hva er vi juridisk og etisk forpliktet til å ta hensyn til?
- **Prosjektplanlegging / vision / risikoanalyse** handler om: Hvordan setter vi retning og håndterer usikkerhet?

Det viktigste er å se at disse **ikke er separate siloer**. De påvirker hverandre hele tiden. Hvis du for eksempel gjør gode brukerundersøkelser, påvirker det brukerhistorier og krav. Kravene påvirker UML-modellene. Modellene påvirker arkitekturvalg. Arkitekturvalg påvirker hva som er lett å teste. Teststrategien påvirker hvilke utviklingsmetoder som passer. Lovkrav og etikk setter grenser for hva som er akseptabelt å lage. Og alt dette må passe inn i prosjektplanlegging og risikostyring.

2. Fra behov til løsning: kjeden fra bruker til kode

En veldig viktig sammenheng i pensum er denne:

Brukerbehov → krav → brukerhistorier / use case → modeller → arkitektur / design → implementasjon → testing / evaluering → forbedring

Dette er egentlig hele systemutvikling i én linje.

Tenk deg at du skal lage en app for timebestilling hos fastlege.

Først må du forstå **brukeren og konteksten**. Dette er MMI og interaksjonsdesign. Hvem er brukerne? Eldre? Unge? Personer med redusert syn? Folk som bruker mobil? Hva prøver de å få gjort raskt og uten feil? Da er fokuset ikke primært teknologi, men **menneskelig måloppnåelse**.

Så oversetter du dette til **krav**. For eksempel: "Brukeren skal kunne bestille time på under 2 minutter", "Løsningen skal fungere med tastatur", "brukeren skal få tydelig tilbakemelding ved feil". Disse kravene er både funksjonelle og ikke-funksjonelle.

Deretter kan du uttrykke behovene som **brukerhistorier**:

"Som pasient vil jeg se ledige timer slik at jeg kan bestille en time som passer meg."

Eller som **use cases**: "Bestille time", "Logge inn", "Avbestille time".

Så modellerer du systemet. I en **domenemodell** kan du vise begreper som Pasient, Lege, Timeavtale, Kalender. I et **sekvensdiagram** viser du hva som skjer når pasienten bestiller time. I et **klassediagram** viser du hvilke klasser systemet kan bestå av. I et **aktivitetsdiagram** viser du prosessflyten. I et **tilstandsmaskindiagram** kan du vise hvordan en timeavtale går fra "ledig" til "reservert" til "bekreftet" til "avbestilt".

Deretter velger du **arkitektur**. Kanskje klient-tjener eller trelagsarkitektur. Kanskje REST-API mellom frontend og backend. Kanskje monolitt hvis systemet er lite, eller mikrotjenester hvis systemet er stort og komplekst.

Så implementerer du. Men hvordan du implementerer påvirkes av metodikk: Scrum, Kanban, XP, TDD, BDD, DevOps.

Til slutt tester og evaluerer du. Merk at her møtes to store deler av pensum:

- **Testing**: fungerer systemet teknisk korrekt?
- **Evaluering / brukertesting**: fungerer systemet godt for brukeren?

Og basert på resultatene går du tilbake og forbedrer systemet. Derfor er interaksjonsdesign og smidig utvikling begge **iterative**.

3. MMI, interaksjonsdesign, prototyping og evaluering: sammenhengen

Disse temaene henger ekstremt tett sammen. De beskriver egentlig én sammenhengende prosess:

Forstå bruker → lag designforslag → bygg prototype → evaluer → forbedre

Interaksjonsdesign handler om å utforme hvordan brukeren samhandler med et system. Målet er ikke bare at systemet "fungerer", men at det oppleves som **intuitivt, effektivt, forståelig og tilfredsstillende**. Derfor kan vi si at interaksjonsdesign er en bro mellom teknologi og psykologi: systemet skal være teknisk mulig, men også menneskelig brukbart.

Prototyping er et verktøy i denne prosessen. En prototype er en tidlig representasjon av løsningen som brukes til å utforske ideer, få tilbakemeldinger og redusere risiko. Sammenhengen er altså at **prototyping brukes for å gjøre interaksjonsdesign konkret og testbart**.

Hvis du designer en innloggingsflyt, kan du først lage en **lavoppløselig prototype** på papir eller i Figma for å utforske struktur og rekkefølge. Senere kan du lage en **høyoppløselig prototype** med mer realistisk interaksjon. Dermed ser du hvordan valg i design faktisk påvirker brukerens opplevelse.

Evaluering er det som forteller deg om designet faktisk er bra. Her er det viktig å rydde i et begrep: notatene dine sier at evaluering er bredere enn testing, og det er riktig. **Testing** er ofte mer fokusert på feil og korrekthet, **evaluering** vurderer verdi, brukskvalitet og om løsningen oppfyller behov. Brukertesting er én type evaluering.

Så sammenhengen er:

- **Interaksjonsdesign** lager forslag
- **Prototyping** konkretiserer forslagene
- **Evaluering** undersøker om de fungerer
- **Brukertesting** er evaluering med faktiske brukere

Et eksempel: Hvis brukerne ofte klikker feil knapp i prototypen, er ikke problemet nødvendigvis en "bug". Det kan være et **interaksjonsdesignproblem** relatert til Don Normans prinsipper: dårlig synlighet, svak feedback, dårlig avbilding, manglende konsistens eller uklare affordances/tilbydelser.

4. Don Norman + universell utforming + WCAG: tre perspektiver på god brukeropplevelse

Dette er et viktig krysspunkt i stoffet.

Don Normans prinsipper handler om **hvordan et grensesnitt blir forståelig og brukbart**. Universell utforming handler om **at flest mulig skal kunne bruke løsningen**. WCAG handler om **konkrete retningslinjer for digital tilgjengelighet**.

Disse tre er altså ikke det samme, men de støtter hverandre.

Hvis du har en knapp for "Send inn", så sier Norman at knappen må være synlig, forståelig og gi feedback. Universell utforming sier at løsningen må kunne brukes av personer med ulike forutsetninger. WCAG gjør dette konkret, for eksempel ved at knappen må kunne brukes med tastatur, ha tilstrekkelig kontrast, tydelig fokusmarkering og forståelig tekst.

Et godt eksamenspoeng er derfor:

- **Norman**: beskriver prinsipper for god interaksjon
- **Universell utforming**: beskriver inkluderende designmål
- **WCAG**: gir konkrete målbare krav for webtilgjengelighet

Eksempel: Et skjema med rød tekst som eneste feilmelding bryter ofte med flere perspektiver samtidig.

- Dårlig **feedback** etter Norman
- Dårlig **forståelig informasjon** i universell utforming
- Kan bryte WCAG fordi informasjon ikke må formidles bare med farge

5. Viktig presisering om universell utforming og lovverk

Her bør notatene dine justeres litt.

Du skriver "Lov om likestilling og forbud mot diskriminering (§18, §19a, §41)" og lister deretter §17, §18 og §41. Her må du være mer presis og konsekvent. Hvis du skal svare på eksamen, bør du heller si noe slikt:

Likestillings- og diskrimineringsloven stiller krav om universell utforming, og for IKT-løsninger er det særlig bestemmelsene om universell utforming av IKT som er sentrale. I tillegg konkretiseres kravene gjennom **forskrift om universell utforming av IKT**, og disse bygger i praksis på WCAG-krav.

Du bør også være forsiktig med å ramse opp paragrafer hvis du ikke er helt sikker på nummereringen. På eksamen er det ofte bedre å si:

- offentlige og private virksomheter rettet mot allmennheten har plikt til universell utforming
- IKT-løsninger som støtter virksomhetens alminnelige funksjoner omfattes
- forskriften konkretiserer kravene
- WCAG brukes som praktisk standard

Det viktigste er å forstå sammenhengen:

Lov → forskrift → tekniske standarder/retningslinjer (WCAG)

WAD, altså Web Accessibility Directive, er relevant fordi EU-regler påvirker kravene som innføres nasjonalt. Men bruk det som **bakgrunn for regelutvikling**, ikke som erstatning for norske lovkrav.

6. UML og modeller: hvordan de henger sammen

En annen hovedsammenheng i pensum er at UML-diagrammer viser **samme system fra ulike perspektiver**.

Mange studenter lærer diagrammene hver for seg, men det riktige er å se dem som et sett av briller:

- **Use case-diagram**: hva brukeren/systemet vil oppnå
- **Domenemodell**: hvilke begreper som finnes i problemområdet
- **Aktivitetsdiagram**: hvilke steg/prosesser som skjer
- **Sekvensdiagram**: hvem som snakker med hvem over tid
- **Klassediagram**: hvordan systemet er bygd statisk
- **Tilstandsmaskindiagram**: hvordan et objekt endrer tilstand
- **Pakkediagram**: hvordan systemet organiseres på høyere nivå

Eksempel med nettbutikk:

Et **use case-diagram** viser at kunden kan søke varer, legge i handlekurv og betale.

En **domenemodell** viser begreper som Kunde, Produkt, Ordre, Betaling.
Et **aktivitetsdiagram** viser prosessen fra "legg i handlekurv" til "bekreftet betaling".
Et **sekvensdiagram** viser meldingene mellom KundeUI, CheckoutController, PaymentService og OrderRepository.
Et **klassediagram** viser klassene og relasjonene.
Et **tilstandsmaskindiagram** kan vise at en ordre går fra Opprettet → Betalt → Sendt → Levert / Kansellert.
Et **pakkediagram** kan gruppere systemet i UI, Domene, Tjenester og Data.

Dette betyr at diagrammene **ikke konkurrerer**, men **utfyller hverandre**.

7. Use case, brukerhistorier og BDD: forskjell og samspill

Dette er et sted mange blander begreper.

En **brukerhistorie** er en kort, smidig formulering av verdi for en bruker:
"Som kunde vil jeg kunne lagre handlekurven slik at jeg kan fortsette senere."

Et **use case** er mer strukturert og beskriver et scenario eller en interaksjon mellom aktør og system, ofte mer detaljert.

BDD bygger videre på dette ved å gjøre krav testbare gjennom atferdsbeskrivelser i naturlig språk, ofte Gherkin.

Sammenhengen kan være slik:

1. Produktansvarlig skriver brukerhistorie
2. Teamet diskuterer akseptkriterier
3. Akseptkriteriene uttrykkes som BDD-scenarier
4. Scenariene kan brukes i akseptansetesting

Eksempel:

Brukerhistorie:
Som kunde vil jeg kunne logge inn slik at jeg får tilgang til mine bestillinger.

BDD-scenario:

- Gitt at jeg er registrert bruker
- Når jeg logger inn med gyldig e-post og passord
- Så skal jeg komme til min kontooversikt

Her ser du samspillet:

- **brukerhistorie** gir mål
- **use case** kan vise interaksjonen
- **BDD** konkretiserer forventet oppførsel
- **akseptansetesting** verifiserer at dette faktisk virker

Presisering: notatene dine sier at akseptkriterier skrives vha domenespesifikt språk "som UML". Det er litt misvisende. UML er ikke et språk for akseptkriterier i BDD-forstand. BDD bruker heller **naturlig språk i strukturert form**, typisk Gherkin. Domenespråk er mer knyttet til at teamet bruker samme begreper om domenet.

8. Domenemodell, DDD og arkitektur

Her er det en veldig fin sammenheng som ofte gir gode eksamenssvar.

Domenemodell i UML beskriver sentrale begreper og relasjoner i problemområdet.
Domain-Driven Design (DDD) er en utviklingstilnærming der programvaren bygges rundt en dyp forståelse av domenet.

Så domenemodellen kan ses som et tidlig uttrykk for den forståelsen DDD ønsker å bygge videre på.

Hvis du lager et system for bibliotek, og i domenemodellen har du Bok, Låner, Utlån, Reservasjon og Gebyr, så hjelper det teamet å snakke presist om problemet. I DDD ønsker man at **koden gjenspeiler domenet**, altså at disse begrepene også får plass i designet og koden. Dette reduserer gapet mellom "forretning" og "teknologi".

Dette påvirker også arkitektur. Hvis du har god domeneforståelse, er det lettere å dele systemet i riktige moduler eller tjenester. Dårlig domeneforståelse gir ofte dårlig arkitektur.

9. Arkitektur: hvordan mønstrene brukes og når

Arkitekturstoffet ditt handler egentlig om ett spørsmål:
Hvordan organiserer vi systemet slik at det håndterer kompleksitet?

Kompleksitetshåndtering er selve begrunnelsen for arkitektur. Når systemer blir store, må vi dele dem opp. Det er her prinsipper som lav kobling og høy kohesjon blir viktige.

Noen viktige sammenhenger:

Standalone → klient-tjener → trelags

Dette kan sees som en utvikling i kompleksitet.

En **standalone-applikasjon** passer når alt er lite og samlet. Når du trenger flere brukere eller sentral datahåndtering, går du mot **klient-tjener**. Når du vil skille ansvar tydeligere, bruker du ofte **trelagsarkitektur**: presentasjon, forretningslogikk og dataaksess.

Dette henger tett sammen med testing:

- Trelagsarkitektur gjør det lettere å enhetsteste logikk uten GUI og database
- Dependency injection og mocking blir lettere når ansvar er tydelig separert

MVC og trelags

Mange blander disse. MVC og trelags er beslektet, men ikke identiske.

- **MVC** er ofte et mønster for å strukturere interaktiv applikasjonslogikk, spesielt i UI-laget
- **trelagsarkitektur** er en bredere lagdeling av hele systemet

Så MVC kan leve **inni** presentasjonslaget i en trelagsarkitektur.

Monolitt vs mikrotjenester

Dette er ikke “gammelt vs moderne”, men ulike kompromisser.

En **monolitt** kan være enklest å utvikle, teste og drifte i starten.

Mikrotjenester gir bedre uavhengig skalering og organisatorisk fleksibilitet, men øker kompleksiteten i distribusjon, logging, feilsøking og datakonsistens.

Krysskobling til DevOps og testing:

- Mikrotjenester krever ofte mer moden **CI/CD**, observability og automatisert testing
- Monolitter kan være enklere med mindre team og begrenset domene

Plugin-arkitektur

Dette passer når du vil ha en stabil kjerne og utvidbar funksjonalitet, som i IDE-er eller nettlesere. Dette henger sammen med **lav kobling** og modularitet.

Broker, event-bus, pipe-filter, peer-to-peer, master-slave

Disse er mer spesialiserte arkitekturmønstre.

- **Broker** brukes når komponenter ikke skal kjenne hverandre direkte
- **Event-bus** brukes når hendelser skal publiseres og håndteres løst koblet
- **Pipe-filter** passer ved databehandlingskjeder
- **Master-slave** passer ved koordinering og replikering
- **Peer-to-peer** passer når noder er mer likeverdige

Eksempel: I et moderne system kan du faktisk kombinere flere:

- frontend/backend via klient-tjener
- backend som mikrotjenester
- intern kommunikasjon via event-bus
- dataanalyse med pipe-filter
- databasekopier med master-replika

Så arkitektur handler ikke om å velge ett mønster for alltid, men om å velge passende struktur for ulike deler.

10. REST som kobling mellom arkitektur og implementasjon

REST passer naturlig inn i klient-tjener- og webarkitektur. Det er altså ikke bare et API-format, men en måte å modellere systemets ressurser på.

Sammenhengen er:

- klienten trenger data og tjenester
- serveren eksponerer ressurser
- REST gir et standardisert grensesnitt mellom dem

Du har skrevet at data og funksjonalitet behandles som ressurser tilgjengelig via URI-er. Det er en god hovedidé. Presiseringen er at **ressurser representerer ting eller konsepter**, mens operasjoner utføres gjennom HTTP-metoder.

For eksempel i en timebestillingsløsning:

- `GET /appointments` henter timeavtaler
- `POST /appointments` oppretter ny avtale
- `PUT /appointments/123` oppdaterer en avtale
- `DELETE /appointments/123` sletter den

REST henger også sammen med testing:

- API-er kan integrasjonstestes
- BDD-scenarier kan ofte formuleres rundt API-oppførsel
- DevOps og CI kan kjøre automatiske API-tester

11. Utviklingsmetodikk: hvordan arbeidet organiseres

Dette stoffet henger sammen som en utvikling i hvordan man tenker om usikkerhet og endring.

Vannfall og UP

Vannfall passer til en verden der krav antas å være stabile og arbeid kan planlegges sekvensielt. Problemet er at programvareutvikling sjelden er så forutsigbar.

UP forsøkte å forbedre dette med iterasjoner, og UML ble sentralt for å modellere systemet.

Smidige metoder

Smidig oppstod som reaksjon mot tunge prosesser og høy endringsmotstand. Det betyr ikke "ingen struktur", men at man aksepterer at læring skjer underveis.

Dette knytter seg tett til interaksjonsdesign:

- interaksjonsdesign er iterativt
- smidig utvikling er iterativ
- begge bygger på tilbakemeldinger og forbedring

Scrum og Kanban

Scrum og Kanban er to måter å styre arbeid på.

Scrum passer når du vil jobbe i tidsbokser med faste seremonier og tydelig sprintmål.
Kanban passer når du vil optimalisere flyt og redusere flaskehalsen uten faste sprinter.

Sammenhengen til testing og leveranse:

- Scrum passer godt med sprintdemo, review og inkrementell levering
- Kanban passer godt med kontinuerlig levering og DevOps-tankegang

XP, TDD og BDD

Dette er mer konkrete kvalitets- og utviklingspraksiser.

- **XP** fokuserer sterkt på teknisk kvalitet, enkelhet, feedback og hyppige leveranser
- **TDD** er en praksis der tester skrives før kode
- **BDD** bygger videre på tanken om at tester også kan uttrykke forventet atferd på et forretningsnært språk

Sammenhengen kan beskrives slik:

- XP gir kultur og praksiser
- TDD styrker kodekvalitet og design
- BDD knytter krav og tester tettere sammen

DevOps

DevOps kobler utvikling og drift. Det er viktig å forstå at DevOps ikke erstatter Agile, men utvider det. Agile handler mye om **hvordan vi utvikler riktig programvare**, mens DevOps handler mye om **hvordan vi leverer og drifter den effektivt og trygt**.

Et godt samspillseksempel:

- Scrum-team utvikler funksjonalitet i små inkremerter
- TDD brukes i utviklingen
- CI kjører automatiske tester ved hver commit/PR
- CD sørger for trygg deploy til test/staging/produksjon
- drift og utvikling samarbeider om logging, overvåkning og feilretting

12. Testing: hvordan hele teststoffet henger sammen

Testing-delen er mye lettere å forstå hvis du deler den i tre hovedspørsmål:

1. **Hva slags feil eller risiko prøver vi å finne?**
2. **På hvilket nivå tester vi?**
3. **Er testingen manuell eller automatisert?**

Verifisering vs validering

Her er en superviktig sammenheng:

- **Verifisering:** bygger vi produktet riktig?

- **Validering:** bygger vi riktig produkt?

Dette kobler testing til evaluering og MMI. En teknisk korrekt løsning kan fortsatt være feil løsning for brukeren.

Testnivåene

- **Enhetstest:** én liten del isolert
- **Integrasjonstest:** samspill mellom deler
- **Systemtest:** hele systemet
- **Akseptansetest:** møter systemet bruker-/kundekrav?

Dette henger også sammen med arkitektur:

- god lagdeling gjør enhetstesting lettere
- løst koblede tjenester krever mer integrasjonstesting
- distribuerte systemer gjør systemtesting vanskeligere

TDD, mocking og dependency injection

Disse tre henger veldig tett sammen.

TDD krever at koden er lett å teste. For at en klasse skal være lett å teste, må avhengighetene kunne isoleres. Derfor bruker man ofte **dependency injection**. Når avhengighetene er injisert, kan de erstattes med **mocks** i testene.

Eksempel: En `OrderService` bruker `PaymentGateway`. I en enhetstest vil du ikke snakke med ekte betalingssystem. Derfor injiserer du en mock av `PaymentGateway` som returnerer kontrollerte svar.

Sammenhengen er altså:

- arkitektur/design for testbarhet
- dependency injection for løs kobling
- mocking for kontrollert testmiljø
- TDD for å styre utvikling gjennom tester

Regresjon, CI og små endringer

Regresjonstesting blir i smidig/DevOps ofte en naturlig konsekvens av at testene kjøres kontinuerlig. Når du gjør små endringer og har CI, blir det lettere å se hva som brøt noe. Derfor henger disse begrepene tett sammen:

- små inkremitter
- hyppige commits/PR-er
- automatiserte tester
- rask feedback
- enklere rollback

Risiko og prioritering

Risk poker, prioritering av testing og testdata henger sammen med prosjektstyring. Du kan ikke teste absolutt alt. Derfor må du prioritere ut fra risiko og konsekvens. Det er egentlig samme tankegang som risikoanalyse i prosjektplanlegging.

Hvis feil i betalingsflyt er kritisk, tester du den mer enn fargevalg på en sekundærsida. Dermed kobles testing til forretningsverdi.

13. Manuell vs automatisk testing: viktig nyansering

Notatene dine for "automatisk eller manuell testing" er litt for grove og bør presiseres.

Det er ikke helt riktig å si at bestemte testtyper "er" automatiske og andre "er" manuelle. Mange testtyper kan være begge deler, avhengig av hvordan de gjennomføres.

Mer presist:

- **Enhetstester** er nesten alltid automatiserte
- **Integrasjonstester** er ofte automatiserte, men kan også være manuelle
- **Systemtester** kan være både manuelle og automatiserte
- **Akseptansetester** kan være begge deler
- **Smoke/sanity** kan være begge deler
- **Usabilitytesting** er ofte manuell
- **Utforskende og destruktiv testing** er typisk manuell, fordi de bygger på menneskelig kreativitet
- **Ytelsetesting** er ofte automatisert med verktøy, men analyseres av mennesker

Det viktigste skillet er egentlig ikke testtypen, men om gjennomføringen kan standardiseres og repeteres effektivt av maskin.

14. Black-box, white-box, usability og destruktiv testing: ulike perspektiver på samme system

Disse begrepene beskriver **hvordan du ser på systemet under testing**.

- **Black-box**: du bryr deg om input og output, ikke intern struktur
- **White-box**: du kjenner intern struktur og tester logikk, veier og kode
- **Usabilitytesting**: du ser på brukeropplevelse og brukervennlighet
- **Destruktiv testing**: du prøver aktivt å få systemet til å feile

Et system kan altså testes fra flere perspektiver samtidig. En innloggingsfunksjon kan:

- enhetstestes white-box av utvikler
- systemtestes black-box av QA
- brukertestes for usability
- angripes destruktivt med rare input eller uventede handlingsmønstre

Dette er et godt eksempel på hvordan pensum overlapper.

15. Prosjektplanlegging, visjon, risikoanalyse og metodikk

Vision-dokument, risikoanalyse, Scrum/Kanban, testing og arkitektur henger sammen fordi de alle handler om **styring av usikkerhet**.

Visjonsdokumentet sier hvor prosjektet skal.

Brukerhistorier og backlog konkretiserer arbeidet.

Risikoanalyse sier hva som kan gå galt.

Arkitekturvalg prøver å redusere teknisk risiko.

Prototyper reduserer bruker- og designrisiko.

Testing reduserer kvalitetsrisiko.

Smidige metoder reduserer planrisiko ved å lære underveis.

Et veldig godt eksamensresonnement er derfor at mange praksiser i faget egentlig er ulike former for **risikoreduksjon**:

- prototype reduserer risiko for feil løsning
 - UML reduserer risiko for misforståelser
 - arkitektur reduserer risiko for kaos ved vekst
 - tester reduserer risiko for feil i produksjon
 - CI/CD reduserer risiko i leveranse
 - universell utforming reduserer risiko for ekskludering og lovbrudd
 - GDPR-arbeid reduserer risiko for personvernbrudd
-

16. GDPR, profesjonsetikk og kvalitet: hvordan de henger sammen

Dette er en veldig viktig del som mange svarer for isolert på eksamen, men som egentlig henger tett sammen med kvalitet og design.

GDPR handler om behandling av personopplysninger og individets rettigheter. Dette er ikke bare "juss ved siden av", men påvirker **hvordan du designer systemet**.

Hvis du bygger en app med persondata, må du tenke:

- hvilke data er faktisk nødvendige?
→ dataminimering
- hvor lenge skal de lagres?
→ lagringsbegrensning
- hvordan sikres de?
→ integritet og konfidensialitet
- hvordan kan brukeren få innsyn eller sletting?
→ registrertes rettigheter

Dette påvirker både arkitektur, databasevalg, tilgangsstyring, logging og UI.

Eksempel: Hvis en bruker skal kunne be om innsyn i egne data, må systemet være bygd slik at data faktisk kan finnes, eksporteres og forklares. Altså påvirker GDPR teknisk design.

Profesjonsetikk går enda bredere. Her er poenget at utvikleren ikke bare har ansvar for å "få det til å virke", men også for konsekvensene av systemet. Det knytter seg til:

- sikkerhet
- personvern
- diskriminering og bias
- brukerens velferd
- samfunnsmessige følger

Derfor henger etikk sammen med kvalitet: et system kan være teknisk imponerende, men fortsatt være dårlig programvare hvis det krenker personvern, ekskluderer brukere eller skaper skade.

17. Konkrete eksempelkjeder: slik anvender du stoffet samlet

Her er noen helhetlige eksempler som viser hvordan du kan bruke flere deler av pensum i ett svar.

Eksempel 1: Lage nettbutikk

Du starter med interaksjonsdesign for å forstå hvordan kunden skal finne varer og betale enkelt. Du lager prototyper og evaluerer dem med brukertesting. Samtidig må du sikre universell utforming og WCAG, slik at løsningen fungerer for tastaturbrukere og personer med nedsatt syn.

Deretter beskriver du systemet med brukerhistorier og use cases. Domenemodellen kan inneholde Kunde, Produkt, Handlekurv og Ordre. Sekvensdiagram viser hvordan checkout fungerer. Klassediagram beskriver struktur.

Arkitekturmessig velger du kanskje klient-tjener med tre lag og REST-API. Hvis systemet er lite, starter du monolittisk. Hvis det vokser mye, kan enkelte deler skilles ut som mikrotjenester, for eksempel betaling og lager.

Utviklingsmessig kan teamet jobbe i Scrum med sprinter og backlog. TDD brukes på kjernelogikk som prisberegning. BDD brukes på akseptansekriterier for kjøpsflyt. CI kjører automatiske tester ved hver PR. CD deplojer til staging. QA gjør systemtesting og usabilitytesting. GDPR er viktig fordi kundenes person- og betalingsdata behandles, så dataminimering, tilgangskontroll og sikker lagring må bygges inn.

Eksempel 2: Bestille legetime

Her blir MMI ekstra viktig, fordi brukerne kan være stressede, eldre eller uvante med teknologi. Norman-prinsipper og universell utforming blir sentrale. Use case: bestille, endre og avbestille time. Tilstandsmaskin for timeavtale. Arkitektur: klient-tjener, kanskje med REST. Testing: enhetstester for validering, integrasjonstester mot API, systemtester for hele flyten, akseptansetester sammen med oppdragsgiver. GDPR er helt sentralt fordi helseopplysninger er sensitive. Profesjonsetikken skjerper ansvaret enda mer.

18. Viktige feil og presiseringer i notatene dine

Her er noen ting jeg ville formulert mer presist til eksamen:

Interaksjonsdesign

Bra hovedinnhold, men skriv heller “interaksjonsmønstre” enn “interaksjonsmønstre”.

De fem dimensjonene i IxD

Den vanlige fremstillingen er:

1. ord
2. visuelle representasjoner
3. fysiske objekter/rom
4. tid
5. atferd

Du har dette omtrent riktig, men “tid-brukt på de første 3 dimensjonene” er litt upresist. Tid handler mer om hvordan interaksjon utfolder seg over tid: animasjon, lyd, sekvenser, respons.

Don Norman

“Tilbydelser” er affordances. God forklaring, men skriv det gjerne som:
“Affordances beskriver hvilke handlinger brukeren oppfatter som mulige.”

Evaluering uten brukere

“Pluralistiske gjennomganger” involverer faktisk mennesker, ofte brukere, utviklere og eksperter sammen. Så den hører ikke helt rent under “ikke involverer brukere” slik du har satt den opp.

Prediktive modeller

Dette er ikke nødvendigvis “maskinlæringsmodeller”. Prediktive modeller i HCI kan også være analytiske modeller som estimerer brukerhandlinger eller ytelse.

Brukerhistorier

Formatet skal være omtrent:
“As a [rolle], I want [mål], so that [nytte].”
Ikke bare “As <> I want<> Suchthat<>”.

Scrum-seremonier

Standup, sprint planning, sprint review og retrospektiv er separate seremonier. I notatet ditt ser sprint review og retrospektiv nesten ut som underpunkter av standup. Det er feil.

TDD og DDD

Du skriver et sted “kontinuerlig og automatisk gjennom DDD.” Der mener du sannsynligvis **TDD**, ikke DDD.

BDD

Akseptkriterier skrives ikke “vha domenespesifikt språk som UML”. Mer riktig: de skrives i et felles, forståelig språk for domene og forretning, ofte som Gherkin-scenarier.

Akseptansetesting

“Er manuell og automatisk” er for kort. Bedre:

“Akseptansetesting kan gjennomføres både manuelt og automatisk.”

Destruktiv testing vs utforskende testing

Her er notatet litt forenklet. Utforskende testing er ikke bare “destruktiv testing, men white-box”. Utforskende testing er bredere og handler om samtidig læring, testdesign og testgjennomføring. Den trenger ikke være destruktiv, men kan være det.

Automatisert testing

“Finner ikke noe eksplisitt” er upresist. Bedre:

“Automatiserte tester er gode på repeterbare, kjente forventninger, men dårlige til å oppdage uventede problemer i brukeropplevelse, estetikk og kreative feilsituasjoner.”

WCAG

Du skriver WCAG 2.0, men i moderne sammenheng brukes ofte 2.1 eller 2.2 i praksis. På eksamen må du likevel svare ut fra pensum, men vær bevisst på at prinsippene er de samme: perceivable, operable, understandable, robust.

19. Hvordan du bør svare anvendt på eksamen

Når du får et spørsmål, ikke svar som en løs definisjon hvis du kan unngå det. Svar heller i denne rekkefølgen:

1. **Definer begrepet kort og presist**
2. **Forklar hva det brukes til**
3. **Forklar hvordan det henger sammen med andre begreper**
4. **Gi et eksempel**
5. **Nevn eventuelle fordeler/ulempes eller typiske misforståelser**

Eksempel hvis du får “Hva er en prototype?”

Dårlig svar: “En prototype er en tidlig versjon av et produkt.”

Sterkere svar:

“En prototype er en tidlig representasjon av et system som brukes for å utforske designvalg og få tilbakemeldinger før full implementasjon. I interaksjonsdesign brukes prototyper for å redusere risiko og evaluere om løsningen støtter brukerens behov. De kan være lavoppløselige eller høyoppløselige, og brukes ofte sammen med brukertesting og evaluering. For eksempel kan en papirprototype av en nettbutikk brukes til å teste om brukeren forstår checkout-flyten før man utvikler backend og database.”

Det er dette sensor ofte oppfatter som ekte forståelse.

20. Den viktigste helhetsforståelsen du bør sitte igjen med

Hvis du skal huske én ting, så er det dette:

Systemutvikling er en iterativ prosess der vi prøver å bygge riktig system på riktig måte.

“Riktig system” handler om:

- brukerbehov
- MMI
- interaksjonsdesign
- evaluering
- brukertesting
- universell utforming
- WCAG
- akseptanse
- etikk
- GDPR

“Riktig måte” handler om:

- UML og modellering
- arkitektur
- smidige metoder

- Scrum/Kanban/XP
- TDD/BDD
- testing
- CI/CD
- DevOps
- risikoanalyse
- prosjektpålegging

Det ene uten det andre er ikke nok. Et system kan være teknisk elegant, men ubrukelig eller ulovlig. Eller det kan være ønsket av brukeren, men teknisk ustabilt og umulig å vedlikeholde. Hele pensumet ditt handler egentlig om å balansere disse hensynene.